

Pipeline de Machine Learning Paralelo para Evaluación de Riesgo Crediticio con Despliegue Continuo en Cloud

Marietha Córdova Delgado
Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

m.cordovad@alum.up.edu.pe

Diego Medina Manrique
Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

dr.medinam@alum.up.edu.pe

Sebastian Guevara Peralta
Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

sa.guevarap@alum.up.edu.pe

Diego Quiñones
Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

da.quinonesv@alum.up.edu.pe

I. INTRODUCCIÓN

La evaluación del riesgo crediticio es fundamental en el sector financiero. Este proceso sirve para determinar si un cliente o prestatario corre el riesgo de incumplir con sus pagos debido a la falta de liquidez en los plazos acordados [1]. Para las empresas prestamistas, esto se traduce en pérdidas económicas, reducción de la rentabilidad y un posible deterioro del patrimonio [2]. Por lo mismo, los prestamistas analizan el historial, capacidad financiera y situación actual de sus clientes para determinar la probabilidad de incumplimiento [3].

En este contexto, el crecimiento de datos financieros en volumen y complejidad, que este sector genera, representa desafíos significativos para los modelos tradicionales [4], que tienen menor precisión y hacen uso de mayores tiempos de cómputo para procesar conjuntos de datos de gran escala [5]. Como consecuencia, se producen retrasos en la generación de resultados y una menor capacidad de respuesta ante los escenarios dinámicos del mercado financiero [6].

La literatura sugiere el uso de algoritmos de Machine Learning sofisticados para el procesamiento de estos datos y el entrenamiento de modelos predictivos orientados a la evaluación del riesgo crediticio [4]. No obstante, el tiempo de cómputo requerido para el entrenamiento sigue siendo elevado, dado que entrenar un modelo con una sola configuración de hiperparámetros puede tomar horas o incluso días, dificultando la exploración de múltiples configuraciones en plazos razonables [7].

En ese sentido, surge la siguiente pregunta de investigación: ¿Cómo reducir el tiempo de entrenamiento de modelos predictivos de Machine Learning sobre conjuntos de datos financieros de alta dimensionalidad mediante el uso de paralelismo de tareas y de datos, y qué ganancia real en términos de *speedup* se obtiene al combinar ambos enfoques en un pipeline integrado?

La Computación en la nube (*Cloud Computing*) resulta una solución viable al ofrecer infraestructura escalable de alto rendimiento que permite ejecutar algoritmos de Machine Learning sin requerir inversión en hardware especializado [8]. Las plataformas en la nube permiten distribuir tanto el entre-

namiento de modelos como la búsqueda de hiperparámetros entre múltiples nodos de forma simultánea, reduciendo significativamente los tiempos de procesamiento [9]. En el sector financiero, esta capacidad resulta especialmente relevante, dado que los modelos de evaluación de riesgo crediticio deben adaptarse continuamente a escenarios de mercado dinámicos y procesar grandes volúmenes de datos en plazos acotados [10].

La integración de paralelismo a nivel de datos mediante OpenMP en un entorno de Computación de Alto Desempeño (HPC), combinada con una infraestructura escalable en la nube gestionada mediante contenedores Docker y pipelines de CI/CD, permitirá reducir significativamente el tiempo de entrenamiento de modelos de Machine Learning para la evaluación de riesgo crediticio, obteniendo un *speedup* medible respecto a enfoques de entrenamiento secuencial tradicional.

El presente proyecto propone abordar las limitaciones computacionales identificadas mediante la combinación de Computación de Alto Desempeño (HPC) y Cloud Computing. Por un lado, el HPC permitirá paralelizar el entrenamiento de modelos de Machine Learning a nivel de datos y de tareas mediante OpenMP, reduciendo los tiempos de procesamiento sobre conjuntos de datos financieros de alta dimensionalidad [9]. Por otro lado, el Cloud Computing proveerá la infraestructura escalable necesaria para distribuir los procesos de entrenamiento y búsqueda de hiperparámetros, eliminando las restricciones físicas del hardware local [8]. Ambos enfoques se integrarán en un pipeline de MLOps automatizado mediante contenedores Docker y metodologías DevOps de CI/CD, lo que garantizará la reproducibilidad, portabilidad y confiabilidad del sistema en entornos de producción [10]. De este modo, la investigación busca demostrar que la combinación de estos paradigmas genera una ganancia real y cuantificable en términos de *speedup*, sin sacrificar la capacidad predictiva de los modelos de evaluación de riesgo crediticio.

El objetivo principal del proyecto es:

Implementar un pipeline de Machine Learning de alto desempeño sobre infraestructura en la nube para la evaluación de riesgo crediticio, combinando paralelismo de tareas mediante HPC con una arquitectura escalable basada en contenedores, con el fin de reducir el tiempo

de entrenamiento y cuantificar el *speedup* obtenido frente a un enfoque secuencial.

En específico, el proyecto busca realizar lo siguiente:

- Diseñar e implementar un pipeline de preprocesamiento y entrenamiento de modelos de Machine Learning sobre conjuntos de datos financieros de alta dimensionalidad.
- Aplicar OpenMP para implementar paralelismo en sistemas de memoria compartida durante el entrenamiento del modelo, evaluando su impacto en el tiempo de cómputo.
- Desplegar la infraestructura de entrenamiento en un entorno de Cloud Computing mediante contenedores Docker, garantizando la portabilidad y escalabilidad del sistema.
- Automatizar el ciclo de vida del modelo mediante un pipeline de CI/CD, asegurando la reproducibilidad del entrenamiento y la confiabilidad del despliegue.
- Medir y comparar el *speedup* obtenido al combinar paralelismo de tareas y de datos frente a la ejecución secuencial.

Finalmente, algunos conceptos claves a considerar en el desarrollo del proyecto son:

- 1) **Computación Paralela y Distribuida:** En lugar de resolver un problema de forma secuencial, este paradigma divide el trabajo en partes más pequeñas que se ejecutan al mismo tiempo en múltiples procesadores. Aplicado al entrenamiento de modelos de ML, permite repartir el conjunto de datos entre varias unidades de cómputo, reduciendo el tiempo total de procesamiento [11].
- 2) **OpenMP** Open Multi-Processing, permite aprovechar los múltiples núcleos de un mismo procesador mediante directivas de compilación en sistemas de memoria compartida. [12].
- 3) **Cloud Computing:** Modelo que permite acceder bajo demanda, a través de internet, a recursos computacionales como servidores, almacenamiento y plataformas de procesamiento, sin necesidad de mantener infraestructura física propia [13].
- 4) **Contenedorización (Docker):** Técnica que empaqueta una aplicación junto con todas sus dependencias en una unidad portátil llamada contenedor, garantizando que el software se ejecute de forma idéntica en cualquier entorno, ya sea local, en servidores o en la nube [14].
- 5) **Integración y Despliegue Continuo (CI/CD):** Conjunto de prácticas DevOps que automatizan las etapas de prueba, validación y entrega del software. En este trabajo, permite que cada actualización del modelo pase automáticamente por un pipeline de verificación antes de ser desplegada en producción, reduciendo errores y tiempos de entrega [15].
- 6) **MLOps (Machine Learning Operations):** Extensión de las prácticas DevOps orientada específicamente al ciclo de vida de los modelos de Machine Learning. Automatiza las etapas de entrenamiento, validación, despliegue y monitoreo del modelo, asegurando que este se

mantenga funcional, escalable y actualizado en entornos de producción [16].

- 7) **LightGBM (Light Gradient Boosting Machine):** Algoritmo de gradient boosting desarrollado por Microsoft que entrena árboles de decisión de forma eficiente mediante una estrategia de crecimiento por hojas (*leaf-wise*) y cálculo de histogramas de características. Su arquitectura permite integración nativa con OpenMP para paralelizar el entrenamiento, haciéndolo especialmente adecuado para conjuntos de datos grandes y de alta dimensionalidad [17].
- 8) **XGBoost (eXtreme Gradient Boosting):** Algoritmo de boosting que construye árboles de decisión de forma secuencial, donde cada árbol corrige los errores del anterior. Incorpora técnicas de regularización para evitar el sobreajuste y soporte nativo para computación paralela, lo que lo hace eficiente y escalable para tareas de clasificación de riesgo crediticio [18].

II. ESTADO DEL ARTE

La predicción del riesgo crediticio mediante algoritmos de aprendizaje automático y la aceleración de pipelines de Machine Learning con computación paralela han sido temas ampliamente estudiados en la literatura reciente. A continuación se presentan los trabajos más relevantes que sustentan las decisiones técnicas adoptadas en este proyecto, organizados en tres grupos temáticos: aplicación de algoritmos de *boosting* al riesgo crediticio, aceleración de modelos mediante OpenMP y paralelismo, y arquitecturas distribuidas para servicios financieros.

A. Algoritmos de boosting aplicados al riesgo crediticio

En Mienye & Sun (2025), se propone un modelo basado en LightGBM combinado con técnicas de IA explicable (SHAP y LIME) para la predicción de incumplimiento en plataformas de *social lending*. Este trabajo es directamente relevante para nuestro proyecto porque valida empíricamente la elección de LightGBM como algoritmo principal en problemas de clasificación binaria sobre datos financieros desbalanceados, y demuestra que es posible obtener buen desempeño predictivo sin sacrificar interpretabilidad, que es un factor crítico en aplicaciones bancarias sujetas a auditorías regulatorias [19].

Además Yu et al. (2024), compara el desempeño de LightGBM, XGBoost, CatBoost y TabNet sobre un dataset bancario de más de 40 000 registros, aplicando reducción de dimensionalidad con PCA y sobremuestreo SMOTEENN. Este estudio resulta especialmente relevante para nuestro proyecto por dos razones: primero, justifica la comparación cabeza a cabeza entre LightGBM y XGBoost que realizamos en la fase experimental; segundo, advierte que la presencia de desbalance de clases exige el uso de técnicas de balanceo internas como las que implementamos a través de los parámetros `is_unbalance` en LightGBM y `scale_pos_weight` en XGBoost. El desbalance también es característica del dataset del presente proyecto, donde solo el 8 % corresponde a la clase positiva [20].

El paper de García-Bedoya et al. (2025), desarrolla un pipeline estructurado que incluye preprocesamiento, ingeniería de características, manejo del desbalance de clases y ajuste de hiperparámetros sobre datasets de préstamos. Aporta a nuestro proyecto al confirmar que el AUC-ROC y el F1-score son las métricas más adecuadas para evaluar el desempeño de modelos de clasificación crediticia con clases desbalanceadas, en lugar de la simple exactitud, que tiende a sobreestimar el desempeño cuando una de las clases domina [21].

B. Paralelismo y aceleración de modelos de boosting

En Ke et al. (2017), describe la arquitectura del framework, en particular su soporte nativo para paralelismo multihilo mediante OpenMP. Esta característica es la base técnica que el presente proyecto explota mediante el parámetro `n_jobs=-1` el cual usa todos los núcleos disponibles del procesador. Sin este soporte nativo, no sería posible alcanzar el speedup observado en nuestros experimentos [22].

En la investigación de Dagum & Menon (1998), se describe a OpenMP como una API estándar para paralelismo en memoria compartida que permite escalar cargas iterativas sin alterar la lógica del algoritmo. Aporta a este proyecto al sustentar la elección de OpenMP, incluido dentro de LightGBM y XGBoost, como la tecnología de paralelismo a nivel de datos, frente a alternativas como MPI que serían más apropiadas para arquitecturas multinodo distribuidas [23].

C. Arquitecturas distribuidas para servicios financieros

En Peng (2022), se propone un modelo de predicción de riesgo bancario sobre arquitectura distribuida Spark+Hadoop integrando los módulos Spark SQL, Spark Streaming y MLlib. Aunque nuestro proyecto no utiliza Spark en su implementación actual, este trabajo es relevante porque sustenta la viabilidad técnica de escalar a arquitecturas multinodo en desarrollos futuros, ofreciendo una hoja de ruta para extender el pipeline a entornos cloud cuando el volumen de datos supere las capacidades de un único nodo [24].

D. Posicionamiento de la propuesta

A partir de esta revisión se identifican dos brechas que el presente trabajo busca cerrar. Primero, la mayoría de los estudios sobre predicción crediticia con LightGBM o XGBoost reporta sus métricas predictivas sin cuantificar explícitamente las ganancias computacionales del paralelismo. Segundo, los trabajos que sí abordan el paralelismo se enfocan principalmente en arquitecturas distribuidas de gran escala (Spark, MPI), descuidando el potencial del paralelismo de tareas sobre una sola máquina, escenario más realista para grupos académicos y pequeñas instituciones financieras. Esta propuesta combina ambos enfoques mediante `joblib` para el paralelismo de tareas y OpenMP para el paralelismo de datos, midiendo de forma explícita el *speedup* obtenido en cada fase del pipeline.

Además, a diferencia de los trabajos revisados, la presente propuesta incorpora prácticas de MLOps mediante Docker, GitHub Actions y despliegue sobre infraestructura cloud en

Azure, permitiendo automatizar el ciclo de vida completo del modelo desde el entrenamiento hasta su despliegue.

III. DISEÑO DEL CASO

A. Descripción del conjunto de datos

1) **Medio de obtención:** El conjunto de datos utilizado es el *Microfinance Loan Dataset*, disponible en Kaggle [25], que corresponde al dataset original de la competencia *Home Credit Default Risk* organizada por Home Credit Group. Los datos están anonimizados y contienen información de solicitudes de préstamos junto con atributos socioeconómicos, financieros y de comportamiento de cada solicitante.

2) **Dimensión del dataset.** El conjunto se distribuye en dos archivos:

- `application_train.csv`, con 307 511 filas y 122 columnas (incluyendo la variable objetivo)
- `application_test.csv`, con 48 744 filas y 121 columnas.

La variable a predecir (TARGET), es binaria, donde 0 indica que los clientes pagaron y 1 significa que son deudores.

La distribución de clases en el dataset de entrenamiento es de aproximadamente 92% para los clientes que pagan frente a 8% de deudores, lo que implica un desbalance crítico. Esto invalida el uso de la exactitud como métrica principal y justifica la elección de AUC-ROC y F1-score.

3) **Análisis preliminar del dataset.**

Se identificó una anomalía documentada en la variable `DAYS_EMPLOYED`: un porcentaje significativo de registros presenta un valor artificial de 365 243 que representa datos faltantes de personas sin empleo formal o jubilados. Estos valores son reemplazados por nulos durante la fase de carga del pipeline.

En cuanto a la calidad general de los datos, el dataset presenta una alta tasa de valores nulos en atributos secundarios relacionados con características del inmueble del solicitante, lo que motivó la implementación de una etapa de limpieza con umbrales configurables (50% para filas, 60% para columnas) y una etapa posterior de imputación diferenciada por tipo de variable. Tras la limpieza el dataset queda con 356 241 registros y 105 columnas, y luego del particionamiento estratificado se obtienen 245 998 registros para entrenamiento, 61 500 para validación y 48 743 para test final.

Considerando la dimensión del dataset (más de 300 000 registros y 122 columnas iniciales), el alto porcentaje de valores faltantes y la necesidad de evaluar múltiples configuraciones de hiperparámetros sobre modelos de *boosting*, el costo computacional del pipeline completo justifica plenamente el uso de técnicas de Computación de Alto Desempeño.

B. Metodología

1) **Descripción general de la Metodología:**

Se aborda un problema de clasificación binaria donde la función mapea un vector de características X_i (datos financieros y socioeconómicos del solicitante) a una variable predictiva $y_i \in \{0, 1\}$ que indica la probabilidad de incumplimiento del préstamo.

El proyecto seguirá un flujo de trabajo (Véase Figura 1) que consta de 5 partes:

- a) **Carga y corrección de anomalías:** Lectura de los archivos CSV y reemplazo del valor artificial 365 243 en `DAYS_EMPLOYED` por nulos.
- b) **Limpieza de nulos excesivos:** Eliminación de filas con más del 50% de valores nulos y columnas con más del 60%.
- c) **Preprocesamiento (imputable en paralelo):** Imputación de variables categóricas con la categoría "Desconocido", imputación y escalado de variables continuas con la mediana y `RobustScaler`, e imputación de variables enteras con la mediana y conversión a tipo `Int64`. Las tres operaciones se ejecutan concurrentemente con `joblib.Parallel`.
- d) **Separación de conjuntos:** División estratificada en entrenamiento (80%) y validación (20%), preservando el conjunto de test final original.
- e) **Entrenamiento paralelo de modelos:** Ejecución simultánea de ocho configuraciones de hiperparámetros (cuatro de `LightGBM` y cuatro de `XGBoost`) mediante `joblib.Parallel` con backend `threading`. Dentro de cada modelo se aprovecha `OpenMP` vía `n_jobs=-1`.

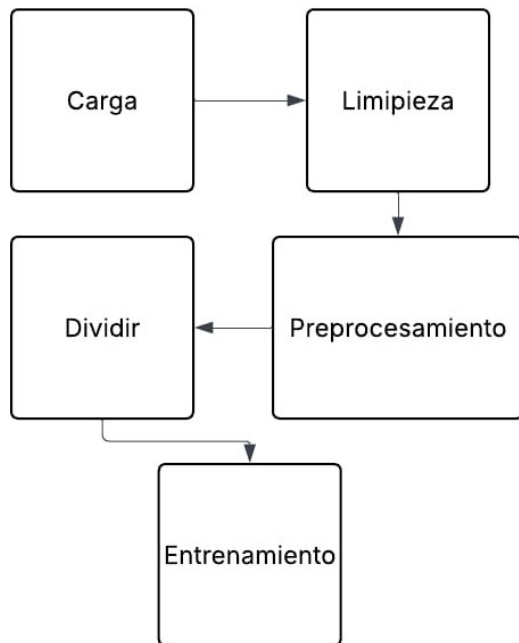


Fig. 1: Flujo del proyecto.

2) Algoritmos seleccionados.

Se eligieron `LightGBM` y `XGBoost` por dos razones técnicas. Primero, ambos integran `OpenMP` de forma nativa para el cálculo paralelo de histogramas, lo que permite explotar el paralelismo a nivel de datos sin requerir bibliotecas adicionales. Segundo, ambos manejan de manera nativa variables categóricas (`enable_categorical=True`), evitando la expansión por *one-hot encoding* que en este dataset generaría más de 240 columnas adicionales.

3) Métricas de rendimiento.

Para evaluar el desempeño predictivo se utiliza el **AUC-ROC**, que mide la calidad de separación entre clases independientemente del umbral de decisión y es robusto frente al desbalance, y el **F1-score**, que pondera precisión y *recall* sobre la clase minoritaria. Para evaluar el desempeño computacional se utiliza el *speedup* (cociente entre tiempo secuencial y paralelo) calculado por fase y de forma global.

4) Estrategia de optimización.

Se identificaron dos niveles complementarios de paralelismo: el paralelismo de tareas, mediante la ejecución concurrente de múltiples configuraciones de modelos, y el paralelismo de datos, mediante `OpenMP` dentro de cada modelo. Esta combinación maximiza el uso de los núcleos disponibles del procesador sin requerir cambios en la lógica de los algoritmos.

5) Implementación cloud.

La solución fue desplegada sobre una máquina virtual Ubuntu 22.04 en Microsoft Azure (Standard D2s_v3), donde se ejecuta el pipeline completo de entrenamiento y despliegue. La infraestructura se encuentra contenerizada mediante `Docker`, garantizando la reproducibilidad del entorno y la portabilidad entre diferentes plataformas.

La aplicación se compone de dos servicios principales gestionados mediante `Docker Compose`: un entorno Jupyter Notebook para experimentación y entrenamiento, y una interfaz web desarrollada en `Streamlit` para inferencia y reentrenamiento de modelos.

Las imágenes de contenedor son almacenadas en `DockerHub`, permitiendo la distribución y actualización centralizada de la aplicación.

Adicionalmente, se implementó un pipeline de integración continua y despliegue continuo (CI/CD) mediante `GitHub Actions`. Este pipeline ejecuta verificaciones automáticas del código y permite lanzar procesos de reentrenamiento que generan nuevas versiones del modelo, las cuales son almacenadas junto con su metadata correspondiente.

Esta arquitectura integra los principios de `Cloud Computing` y `MLOps`, proporcionando un entorno reproducible, escalable y automatizado para el ciclo de vida del modelo.

IV. EXPERIMENTACION Y RESULTADOS

A. Configuración experimental

1) Hardware utilizado.

Los experimentos fueron ejecutados sobre una máquina con: Procesador de 2 núcleos físicos y 4 núcleos lógicos, 16.8 GB de memoria RAM, Sistema operativo Ubuntu Linux 22.04, Python 3.11, LightGBM 4.6.0, XGBoost 3.2.0, y Scikit-learn 1.9.0

2) Configuraciones de hiperparámetros.

Se definieron ocho configuraciones (cuatro por algoritmo) variando `learning_rate`, `num_leaves` para LightGBM y `max_depth` para XGBoost (Véase Tabla I).

TABLE I: Configuraciones de hiperparámetros evaluados.

Config_id	Hiperparámetros	Modelo
LGB-1	lr=0.05, num_leaves=31	LightGBM
LGB-2	lr=0.05, num_leaves=63	LightGBM
LGB-3	lr=0.10, num_leaves=31	LightGBM
LGB-4	lr=0.01, num_leaves=127	LightGBM
XGB-1	lr=0.05, num_leaves=6	XGBoost
XGB-2	lr=0.05, num_leaves=8	XGBoost
XGB-3	lr=0.10, num_leaves=6	XGBoost
XGB-4	lr=0.01, num_leaves=10	XGBoost

3) **Diseño de los experimentos.** El pipeline completo se ejecutó dos veces sobre el mismo dataset preprocesado: una en modo secuencial (los ocho modelos uno tras otro, con `n_jobs=1`) y otra en modo paralelo (los ocho modelos simultáneamente, cada uno con `n_jobs=-1`). Se cronometraron por separado las cinco fases del pipeline para identificar dónde se concentra la ganancia del paralelismo.

B. Resultados de desempeño predictivo

De la evaluación del AUC-ROC y el F1-score obtenidos por cada configuración sobre el conjunto de validación (Véase Figura II) se puede ver que la configuración con mejor desempeño predictivo fue **LGB-1** (LightGBM con `learning_rate=0.05` y `num_leaves=31`), con un AUC-ROC de 0.7597. Se observa que las cuatro configuraciones de LightGBM superan a las cuatro de XGBoost en AUC-ROC, lo cual es consistente con los hallazgos reportados por Yu et al. [20] sobre la superioridad de LightGBM en escenarios de datos desbalanceados.

C. Resultados de desempeño computacional

En la Figura III se compara los tiempos de ejecución por fase entre los modos secuencial y paralelo, y el *speedup* obtenido en cada caso. Además, en las Figuras 2 y 3 se muestra la comparación de los tiempos de ejecución de manera

TABLE II: Resultados de las ocho configuraciones evaluadas.

Config_id	AUC-ROC	F1-score
LGB-1	0.759701	0.272966
LGB-2	0.759121	0.278016
LGB-3	0.758506	0.270140
LGB-4	0.759365	0.283655
XGB-1	0.755748	0.278735
XGB-2	0.747241	0.278735
XGB-3	0.753827	0.278340
XGB-4	0.739068	0.278931

gráfica. La fase de entrenamiento concentra la mayor ganancia de paralelización.

TABLE III: Tiempos de ejecución y *speedup* por fase del pipeline.

Fase	Sec.(s)	Par.(s)	Speedup
Carga	3.85	3.18	1.21x
Limpieza	1.49	1.65	0.9x
Preprocesamiento	6.79	8.11	0.84x
Split	0.90	0.97	0.93x
Entrenamiento	878.90	251.25	3.5x
TOTAL	891.95	265.21	3.36x

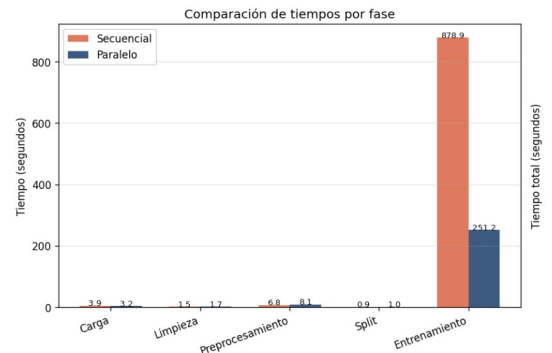


Fig. 2: Comparación por fases

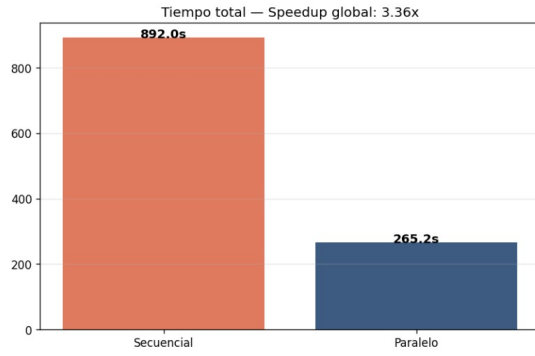


Fig. 3: Comparación por tiempo total

D. Evaluación del paralelismo de tareas

El paralelismo de tareas se implementó mediante `joblib.Parallel` con el backend `threading`, ejecutando las ocho configuraciones de hiperparámetros de forma concurrente. En la ejecución secuencial, los modelos se entrenan uno tras otro con `n_jobs=1`, acumulando tiempos individuales. En la ejecución paralela, los ocho modelos se lanzan simultáneamente, de modo que el tiempo total queda determinado por el modelo más lento.

La Figura 4 muestra la comparación de tiempos individuales por configuración entre la ejecución secuencial (`n_jobs=1`) y la paralela (`n_jobs=4`). Se observa que modelos de menor costo computacional como XGB-3 finalizaron en aproximadamente 79 s en paralelo, mientras que el modelo más costoso, LGB-4, determinó el tiempo total de la fase con 251 s. Esto confirma que el bottleneck en la ejecución paralela es la configuración más exigente, tal como predice la Ley de Amdahl para tareas de distinta duración.

El speedup obtenido en la fase de entrenamiento fue de **3.50x** (de 878.90 s a 251.25 s), significativamente inferior al speedup ideal de 8x esperado para ocho tareas concurrentes. Esta brecha se explica por la contención de recursos: cuando los ocho modelos se ejecutan en paralelo, cada uno solicita todos los núcleos disponibles vía OpenMP, generando competencia por los mismos recursos físicos en una máquina con solo 2 núcleos físicos.

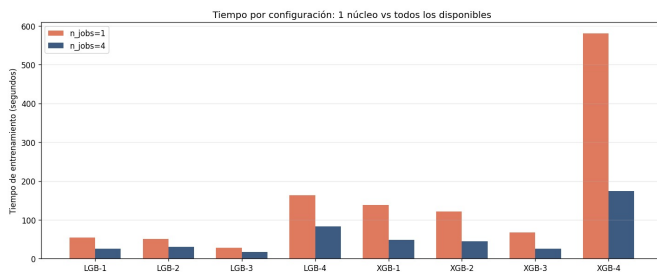


Fig. 4: Tiempo de entrenamiento por configuración: secuencial vs. paralelo.

E. Evaluación del paralelismo de datos

El paralelismo de datos se implementó a través del soporte nativo de OpenMP en LightGBM y XGBoost, controlado mediante el parámetro `n_jobs`. Con `n_jobs=-1`, cada modelo utiliza todos los núcleos lógicos disponibles para distribuir el cálculo de histogramas de características, operación central en los algoritmos de *gradient boosting*.

La interacción entre el paralelismo de tareas (ocho modelos concurrentes) y el paralelismo de datos (cada modelo con `n_jobs=-1`) generó un doble nivel de paralelismo. Sin embargo, en una máquina con 2 núcleos físicos (4 lógicos), cuando los ocho modelos compiten simultáneamente por los mismos recursos vía OpenMP, se produce contención. Esto se evidencia en los tiempos individuales: modelos que en secuencial tomaban entre 26 s y 73 s, en paralelo escalonaron hasta 79–251 s debido a dicha contención.

En entornos con mayor cantidad de núcleos físicos, como los disponibles en instancias cloud de mayor tamaño (por ejemplo, Standard_D4s_v3 con 4 núcleos físicos), este speedup debería incrementarse de forma significativa, ya que la contención se reduciría proporcionalmente.

F. Análisis de escalabilidad

Con el objetivo de evaluar el impacto del paralelismo de datos sobre el desempeño computacional del pipeline, se realizaron experimentos variando la cantidad de núcleos disponibles para el entrenamiento de los modelos. Para ello, se configuró el parámetro `n_jobs` con valores de 1, 2 y 4, aprovechando el soporte nativo de OpenMP incorporado en LightGBM y XGBoost.

La escalabilidad se evaluó mediante la medición de los tiempos de entrenamiento y el cálculo del *speedup* respecto a la ejecución con un único núcleo, utilizando la expresión:

$$Speedup(n) = \frac{T_1}{T_n}$$

donde T_1 corresponde al tiempo de ejecución utilizando un núcleo y T_n representa el tiempo obtenido con n núcleos.

La Figura 5 muestra la evolución del tiempo de ejecución a medida que aumenta el número de núcleos disponibles. Se observa una reducción progresiva del tiempo de entrenamiento al incrementar los recursos de cómputo, confirmando que los algoritmos aprovechan efectivamente el paralelismo de memoria compartida proporcionado por OpenMP.

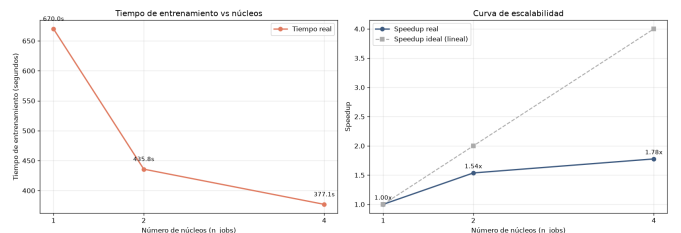


Fig. 5: Análisis de escalabilidad del pipeline para diferentes cantidades de núcleos.

La Figura 6 presenta el comportamiento del mejor modelo (LGB-1). Los resultados evidencian que el beneficio al incrementar el número de núcleos no es estrictamente lineal, debido a la existencia de costos de sincronización, gestión de hilos y contención de recursos compartidos.

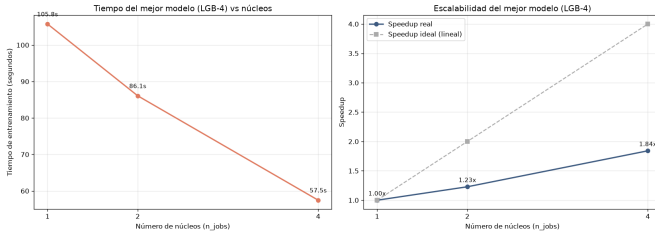


Fig. 6: Escalabilidad del mejor modelo (LGB-1) identificado durante la experimentación.

La Tabla IV resume los tiempos y speedups obtenidos por número de núcleos para el total del pipeline de entrenamiento.

TABLE IV: Tiempos de entrenamiento y speedup por número de núcleos.

n_jobs	Tiempo (s)	Speedup real	Speedup ideal
1	1201.60s	1.00x	1.0x
2	605.65s	1.98x	2.0x
4	448.96s	2.68x	4.0x

G. Despliegue en la nube e interfaz de usuario

1) *Infraestructura en Microsoft Azure*: La solución fue desplegada en una máquina virtual Azure Standard D2s_v3 (Ubuntu 22.04 LTS, 2 vCPUs, 8 GB RAM). La instalación del entorno se realizó mediante un script `docker_bash.sh` que automatiza la instalación de Docker y Docker Compose. El repositorio fue clonado con soporte para Git LFS, necesario para gestionar los datasets de más de 100 MB.

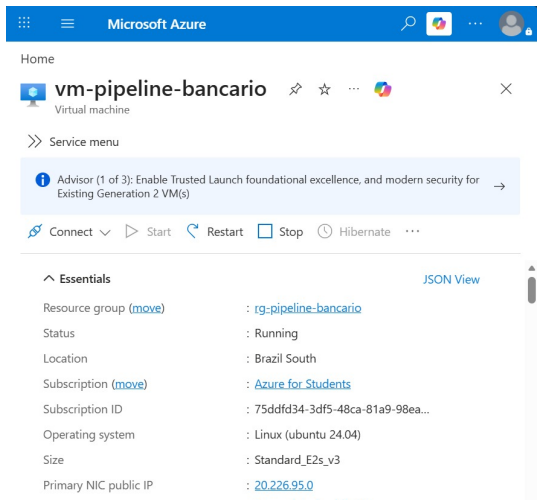


Fig. 7: Máquina virtual en ejecución en Microsoft Azure.

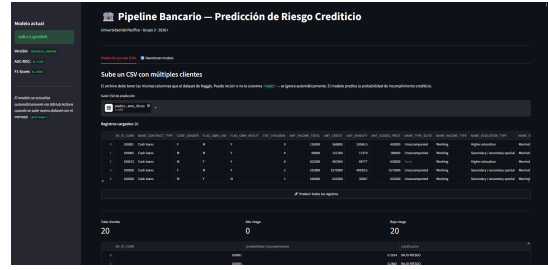


Fig. 8: Streamlit - Modelo y Resultados de Predicción.

Los servicios del proyecto son accesibles desde cualquier navegador mediante las direcciones <http://20.226.95.0:8888> (Jupyter Notebook) y <http://20.226.95.0:8501> (interfaz Streamlit).

2) *Interfaz Streamlit*: Se desarrolló una interfaz web con dos funcionalidades principales:

Predicción por lote: El usuario sube un archivo CSV con datos de solicitantes. El sistema aplica automáticamente el mismo preprocesamiento utilizado durante el entrenamiento y genera predicciones de probabilidad de incumplimiento para cada registro, clasificándolos en “ALTO RIESGO” o “BAJO RIESGO”. Los resultados se pueden descargar en formato CSV.

Reentrenamiento: El usuario sube un nuevo archivo CSV de entrenamiento (con columna TARGET). El sistema reemplaza el dataset de entrenamiento, realiza un commit automático al repositorio de GitHub con el mensaje [entrenar], y dispara automáticamente el pipeline de GitHub Actions. Este pipeline entrena los ocho modelos, selecciona el mejor por AUC-ROC y hace commit del nuevo `modelo_actual.pkl` al repositorio, completando el ciclo de despliegue continuo.

3) *Pipeline CI/CD con GitHub Actions*: El pipeline de integración y despliegue continuo se implementó en `.github/workflows/ci.yml` con dos jobs:

El job `verificacion` se ejecuta en cada push a main, instala las dependencias, verifica que las librerías clave (LightGBM, XGBoost, joblib) se importan correctamente y realiza el build de la imagen Docker.

El job `entrenamiento` se activa únicamente cuando el mensaje del commit contiene la etiqueta [entrenar] o cuando se dispara manualmente. Ejecuta el script `train_and_export.py`, que realiza el análisis completo de escalabilidad, selecciona el modelo óptimo y exporta `modelo_actual.pkl` y `modelo_metadata.json`. Finalmente, realiza un commit automático al repositorio con estos archivos, bajo el mensaje “modelo actualizado automáticamente [skip ci]”.

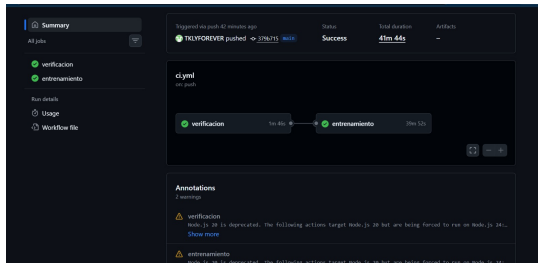


Fig. 9: Pipeline CI/CD completado exitosamente en GitHub Actions.

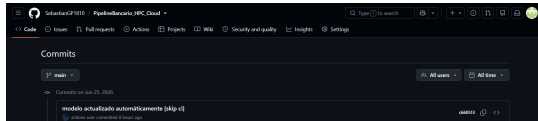


Fig. 10: Commit automático del modelo por GitHub Actions Bot.

El código del proyecto se encuentra disponible en el repositorio público: github.com/SebastianGP1810/PipelineBancario_HPC_prediction. La imagen Docker está publicada en DockerHub como `rodie916/pipeline-bancario:latest`.

V. DISCUSIÓN

Los resultados confirman la hipótesis planteada: la combinación de paralelismo de tareas y datos reduce significativamente el tiempo de ejecución del pipeline, alcanzando un speedup global de **3.36x** (de 891.95 s a 265.21 s). La ganancia se concentra casi en su totalidad en la fase de entrenamiento, que pasó de **878.90 s a 251.25 s**, logrando un speedup de **3.50x**.

Este resultado se explica por la ejecución concurrente de los ocho modelos, donde el modelo más lento (LGB-4, 251.20 s en paralelo frente a 299.37 s en secuencial) determina el tiempo de la fase, mientras que modelos más ligeros como XGB-3 finalizaron en apenas 79.18 s.

Las fases de carga y split presentan speedups cercanos a 1 (1.21x y 0.93x respectivamente), lo cual era esperable dado que estas operaciones están limitadas por entrada/salida de disco. Las fases de limpieza (0.90x) y preprocesamiento (0.84x) muestran una ligera regresión atribuible a la variabilidad del sistema operativo y la sobrecarga de gestión de hilos en tareas de muy corta duración.

Es importante destacar que el speedup en entrenamiento (3.50x) es inferior al speedup ideal lineal (8x para ocho tareas). Esta diferencia se explica por la contención de recursos: cuando los ocho modelos se ejecutan simultáneamente, cada uno solicita todos los núcleos disponibles vía OpenMP, generando competencia por los mismos 2 núcleos físicos. En entornos con mayor cantidad de núcleos, este speedup debería incrementarse.

Cuellos de botella y limitaciones:

- *Contención de OpenMP*: 8 modelos compiten por 2 núcleos físicos, limitando el speedup real vs. ideal.
- *Overhead de joblib*: Las fases de limpieza y preprocesamiento muestran speedup menor a 1 porque el trabajo es demasiado pequeño relativo al costo de lanzar hilos concurrentes.
- *Git LFS*: Los datasets de entrenamiento superan los 100 MB, requiriendo Git LFS para el versionado, lo que introduce latencia en el pipeline de CI/CD.
- *Compatibilidad de versiones*: LightGBM 4.6 es estricto con el matching de `pandas_categorical` entre entrenamiento e inferencia, requiriendo reproducir el preprocesamiento completo para la predicción.

¿Cómo podría mejorarse la implementación?

- Usar una VM con más núcleos físicos (Standard_D4s o mayor) para reducir la contención de OpenMP y acercar el speedup al valor ideal.
- Implementar un caché del preprocesamiento para no reprocesar el train completo en cada predicción de Streamlit.
- Separar el contenedor de entrenamiento del de inferencia, eliminando la dependencia del tren en tiempo de predicción.
- Incorporar MLflow para el tracking de experimentos y versionado de modelos, reemplazando el archivo `modelo_metadata.json` con una solución más robusta.

Dificultades encontradas:

- Compatibilidad entre `pandas 3.x` y `streamlit`, resuelta utilizando `streamlit>=1.40.0` en lugar de una versión fija.
- El mismatch de columnas categóricas entre entrenamiento e inferencia causado por la detección automática de tipos basada en cardinalidad relativa, que varía según el tamaño del dataset. Resuelto usando directamente las `pandas_categorical` almacenadas en el modelo.
- El proxy de GitHub Codespaces limita los uploads a más de 200 MB, lo que obligó a migrar el despliegue a la VM de Azure.
- La autorización de GitHub Actions Bot para hacer push al repositorio requirió configurar un Personal Access Token (PAT) como secret del repositorio.

En términos de calidad predictiva, el mejor modelo fue LGB-1 (LightGBM) con **AUC = 0.7597** y **F1 = 0.2730**, idéntico entre la versión secuencial y paralela, confirmando que la paralelización no afecta la reproducibilidad de los modelos entrenados.

VI. CONCLUSIONES Y TRABAJOS FUTUROS

A. Síntesis de hallazgos principales

Este proyecto demostró que la combinación de paralelismo de tareas (`joblib`) y paralelismo de datos (OpenMP) en un pipeline de Machine Learning genera una ganancia computacional real y cuantificable, alcanzando un speedup global de

3.36x en el tiempo total del pipeline y de **3.50x** en la fase de entrenamiento. Los principales hallazgos son:

- 1) **Eficacia del paralelismo combinado:** La ejecución concurrente de ocho configuraciones de modelos mediante `joblib.Parallel`, combinada con el uso de todos los núcleos disponibles dentro de cada modelo vía OpenMP, redujo el tiempo de entrenamiento de 878.90 s a 251.25 s.
- 2) **Calidad predictiva mantenida:** LGB-1 (LightGBM con `learning_rate=0.05` y `num_leaves=31`) fue la configuración óptima con AUC-ROC de 0.7597, resultado idéntico en modo secuencial y paralelo, confirmando que la paralelización no afecta la reproducibilidad.
- 3) **Limitación de hardware:** Con solo 2 núcleos físicos, el speedup real (3.50x) se aleja del ideal (8x) debido a la contención de recursos. El análisis de escalabilidad [1, 2, 4] confirma que el beneficio del paralelismo crece progresivamente pero de forma no lineal.
- 4) **Pipeline MLOps funcional:** Se implementó un ciclo de vida completo del modelo: desde el entrenamiento y exportación del modelo óptimo, hasta su despliegue automático mediante GitHub Actions y su uso en predicción en tiempo real a través de Streamlit.
- 5) **Despliegue cloud reproducible:** La containerización con Docker y el despliegue en Azure garantizan que el entorno es reproducible en cualquier máquina, eliminando dependencias del hardware local.

B. Recomendaciones para implementaciones futuras

- 1) **Escalado horizontal en Azure Kubernetes Service (AKS):** Migrar de un único contenedor a un clúster de pods en AKS permitiría distribuir el entrenamiento entre múltiples nodos, acercando el speedup al ideal y habilitando la evaluación de arquitecturas MPI para paralelismo distribuido.
- 2) **Incorporación de MLflow:** Reemplazar el archivo `modelo_metadata.json` con un servidor MLflow permitiría el tracking completo de experimentos, comparación de versiones y rollback automático ante degradación del modelo.
- 3) **Técnicas avanzadas de balanceo de clases:** Explorar SMOTE, ADASYN o pesos de clase adaptativos para mejorar el F1-score sobre la clase minoritaria, actualmente limitado a 0.27 a pesar del AUC-ROC de 0.76.
- 4) **Monitoreo de data drift:** Implementar detectores de distribución (por ejemplo, Evidently o NannyML) que alerten automáticamente cuando la distribución de los datos de entrada cambie significativamente respecto al dataset de entrenamiento, disparando el pipeline de reentrenamiento.
- 5) **Separación de contenedores de entrenamiento e inferencia:** En producción, el contenedor de inferencia no debería depender del dataset de entrenamiento completo. Una arquitectura de microservicios con endpoints de

predicción independientes reduciría la latencia y el uso de memoria.

REFERENCES

- [1] A. L. Leal Fica, M. A. Aranguiz Casanova y J. Gallegos Mardones, "Análisis de riesgo crediticio, propuesta del modelo Credit Scoring," Revista Facultad de Ciencias Económicas: Investigación y Reflexión, vol. XXVI, no. 1, pp. 181–207, 2018. DOI: <https://doi.org/10.18359/rfce.2666>
- [2] M. Jumbo y R. Chávez, "Rentabilidad, capital y riesgo crediticio en bancos ecuatorianos," Contaduría y Administración, vol. 66, no. 1, 2021. [En línea]. Disponible en: https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S2448-76782021000100002.
- [3] Allianz Trade, "How to Improve Credit Risk Analysis," Allianz Trade Insights, [En línea]. Disponible en: https://www.allianz-trade.com/en_US/insights/how-to-improve-credit-risk-analysis.html
- [4] Y. Liu et al., "Credit Scoring Prediction Using Deep Learning Models in the Financial Sector," IEEE Access, 2025. [En línea]. Disponible en: <https://ieeexplore.ieee.org/document/11087204>
- [5] Y. Zhang et al., "Integration of CNN Models and Machine Learning Methods in Credit Score Classification," Computational Economics, 2025. [En línea]. Disponible en: <https://link.springer.com/article/10.1007/s10614-025-10893-5>
- [6] J. Henríquez et al., "Machine Learning for Enhanced Credit Risk Assessment: An Empirical Approach," Journal of Risk and Financial Management, vol. 16, no. 12, p. 496, 2023. [En línea]. Disponible en: <https://www.mdpi.com/1911-8074/16/12/496>
- [7] B. Bischl et al., "Hyperparameter Optimization: Foundations, Algorithms, Best Practices and Open Challenges," arXiv preprint, arXiv:2107.05847, 2021. [En línea]. Disponible en: <https://arxiv.org/pdf/2107.05847>
- [8] L. Li et al., "A System for Massively Parallel Hyperparameter Tuning," arXiv preprint, arXiv:1810.05934, 2020. [En línea]. Disponible en: <https://arxiv.org/abs/1810.05934>
- [9] A. Johnson y M. McCourt, "Orchestrate: Infrastructure for Enabling Parallelism during Hyperparameter Optimization," arXiv preprint, arXiv:1812.07751, 2018. [En línea]. Disponible en: <https://arxiv.org/pdf/1812.07751>
- [10] Y. Li et al., "SpotTune: Leveraging Transient Resources for Cost-efficient Hyper-parameter Tuning in the Public Cloud," arXiv preprint, arXiv:2012.03576, 2020. [En línea]. Disponible en: <https://arxiv.org/pdf/2012.03576>
- [11] V. Hegde y S. Usmani, "Parallel and Distributed Deep Learning," Stanford University, CME 323, 2016. [En línea]. Disponible en: https://web.stanford.edu/rezab/classes/cme323/S16/projects_reports/hedge_usmani.pdf
- [12] A. Hager y G. Wellein, "Parallel Paradigms in Modern HPC: A Comparative Analysis of MPI, OpenMP, and CUDA," arXiv preprint, arXiv:2506.15454, 2025. [En línea]. Disponible en: <https://arxiv.org/pdf/2506.15454>
- [13] P. Mell y T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, NIST Special Publication 800-145, 2011. [En línea]. Disponible en: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>
- [14] Docker Inc., "What is a Container?," Docker Documentation, 2024. [En línea]. Disponible en: <https://www.docker.com/resources/what-container/>
- [15] IBM, "What Are CI/CD and the CI/CD Pipeline?," IBM Think, 2024. [En línea]. Disponible en: <https://www.ibm.com/think/topics/ci-cd-pipeline>
- [16] D. Symeonidis et al., "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," arXiv preprint, arXiv:2205.02302, 2022. [En línea]. Disponible en: <https://arxiv.org/pdf/2205.02302>
- [17] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017. [En línea]. Disponible en: <https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf>

- [18] T. Chen y C. Guestrin, "XGBoost: A Scalable Tree Boosting System," en Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794. DOI: <https://doi.org/10.1145/2939672.2939785>
- [19] I. D. Mienye & Y. Sun, "Explainable AI based LightGBM prediction model to predict default borrower in social lending platform", *Intelligent Systems with Applications*, vol.26, p.200441, 2025. DOI: <https://doi.org/10.1016/j.iswa.2025.200441>
- [20] C. Yu et al., "Advanced User Credit Risk Prediction Model using LightGBM, XGBoost and Tabnet with SMOTEENN", *arXiv preprint arXiv:2408.03497*, nov. 2024. [En línea]. Disponible en: <https://arxiv.org/abs/2408.03497>.
- [21] O. García-Bedoya et al., "Data-Driven Loan Default Prediction: A Machine Learning Approach for Enhancing Business Process Management", *Systems*, vol.13, no.7, p.581, jul.2025. DOI: <https://doi.org/10.3390/systems13070581>
- [22] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", en *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol.30, 2017. [En línea]. Disponible en: <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>.
- [23] L. Dagum y R. Menon, "OpenMP: An Industry-Standard API for Shared-Memory Programming", *IEEE Computational Science and Engineering*, vol.5, no.1, pp.46–55, ene.1998. DOI: <https://doi.org/10.1109/99.660313>
- [24] W. Peng, "Bank Financial Risk Prediction Model Based on Big Data", *Scientific Programming*, vol.2022, p.3398545, feb.2022. DOI: <https://doi.org/10.1155/2022/3398545>
- [25] Y. Daniel, "Microfinance Loan Dataset", Kaggle, 2023. [En línea]. Disponible en: <https://www.kaggle.com/datasets/youngdaniel/loan-dataset>.